PYTHON UNIT 5 QB – (4 marks)

1. Define Object-Oriented Programming. List any two advantages.

1. Object-Oriented Programming (OOP) is a programming method that uses classes and objects to organize code. It represents real-world entities with data and methods together.
2. In OOP, objects contain attributes and methods that define their behavior and properties. This approach makes programs easier to understand and manage.
3. OOP supports modular programming by dividing the program into smaller reusable parts. Each class works like a blueprint for creating objects.
4. One advantage of OOP is code reusability through inheritance. Existing classes can be reused to reduce duplication of code.
5. Another advantage is better maintainability because related data and functions stay together. This helps programmers easily update and debug programs.
6. OOP also improves scalability and flexibility in large applications. Features like polymorphism and encapsulation make the program more secure and organized.

2. What is a class and an object in Python? Give one example.

1. A class in Python is a blueprint used to create objects with similar properties and methods. It defines the structure and behavior of objects.
2. An object is an instance of a class that contains its own data and methods. Objects are created from the class whenever needed.
3. Classes help organize data and functions together in a single unit. This makes programming more modular and reusable.
4. Objects represent real-world entities and can perform actions using methods. Each object can have different values for its attributes.
5. For example, a class named Dog can have attributes like name and age. An object like dog1 = Dog("Buddy",3) represents a specific dog.
6. In this example, Dog is the class and dog1 is the object created from it. The object can access attributes such as name, age, and species.

3. Explain the concept of encapsulation with a simple example.

1. Encapsulation is a process of combining data and methods into a single unit called a class. It helps protect data from direct access outside the class.
2. In encapsulation, important variables are kept private and can only be accessed through methods. This improves security and control over data.
3. Encapsulation also improves maintainability and readability of programs. It allows programmers to change implementation details without affecting other parts.
4. Python uses private members with double underscore prefixes like __salary. These members cannot be directly accessed from outside the class.
5. For example, an Employee class may contain a private variable called __salary. A public method like show_salary() is used to display the salary safely.
6. If someone tries to access emp.__salary directly, Python gives an error. This shows how encapsulation protects internal data from unauthorized access.

4. What are Python namespaces? Explain briefly.

1. A namespace in Python is a container that stores names of variables, functions, and objects uniquely. It prevents name conflicts in a program.
2. Python provides different types of namespaces to manage identifiers properly. These namespaces help organize variables according to their scope.
3. The built-in namespace contains predefined functions like print() and len(). It is created automatically when the Python interpreter starts.
4. The global namespace contains names defined in the main program or module. It remains active until the program execution ends.
5. The local namespace is created whenever a function is called. It stores variables and arguments defined inside that function only.
6. Namespaces improve program organization and avoid collisions between variable names. They make Python programs easier to manage and debug.

5. Define scope in Python. List different types of scopes.

1. Scope in Python refers to the area where a variable or name can be accessed in a program. It decides the visibility and lifetime of variables.
2. Python follows the LEGB rule while searching for variables in a program. The order is Local, Enclosing, Global, and Built-in scope.
3. Local scope contains variables defined inside a function. These variables can only be accessed within that function.
4. Enclosing scope exists in nested functions where inner functions can access variables of outer functions. It is useful in closures and nested programming.
5. Global scope contains variables declared outside all functions in the main program. These variables can be accessed throughout the module.
6. Built-in scope contains predefined Python names like range(), open(), and print(). These names are available everywhere in the program automatically.

6. What is a constructor? Write syntax of init() method.

1. A constructor is a special method in Python that is executed automatically when an object is created. It is mainly used to initialize instance variables of the object.
2. In Python, the constructor method is named init(). This method runs once for every object created from the class.
3. Constructors help assign initial values to object properties during object creation. Without constructors, values must be assigned manually each time.
4. The constructor can take arguments along with self to initialize object data. The self keyword refers to the current object of the class.
5. Constructors improve code organization and make object creation easier. They are optional because Python provides a default constructor if not defined.
6. Syntax of init() method is shown below:

```
class Student:
  def __init__(self, name):
    self.name = name
```

7. Differentiate between local and global scope.

1. Local scope contains variables declared inside a function or method. Global scope contains variables declared outside all functions in the program.
2. Local variables can only be accessed within the function where they are created. Global variables can be accessed throughout the module or program.
3. Local scope exists only during function execution. Global scope remains active until the program execution finishes.
4. Changes made to local variables affect only that specific function. Changes made to global variables can affect the entire program.
5. Python searches for local variables first before checking global variables. This follows the LEGB rule used in Python scope resolution.
6. Example of local variable is a variable inside a method like a=1000. Example of global variable is x=10 declared outside the function and used everywhere.

8. What is inheritance? Mention its types.

1. Inheritance is a feature of Object-Oriented Programming where one class acquires properties and methods of another class. It helps in code reuse and hierarchical classification.
2. The class that provides properties is called the parent or base class. The class that inherits properties is called the child or derived class.
3. Inheritance reduces code duplication because common features can be written once in the parent class. It also makes programs easier to maintain.
4. Single inheritance is a type where one child class inherits from one parent class. It is the simplest form of inheritance.
5. Multiple inheritance allows one child class to inherit from more than one parent class. Python also supports multilevel inheritance and hierarchical inheritance.
6. Hybrid inheritance is a combination of different inheritance types in one program. Python uses Method Resolution Order (MRO) to manage hybrid inheritance.

9. Define polymorphism in Python with a simple example.

1. Polymorphism means “many forms” where the same method or function behaves differently in different situations. It makes programs more flexible and reusable.
2. In Python, polymorphism allows the same method name to perform different actions depending on the object. This behavior is mainly achieved through method overriding.
3. Runtime polymorphism happens when a child class provides its own version of a parent class method. The method behavior is decided during program execution.
4. Polymorphism is useful in real-world applications like payment systems with different payment methods. The same function name can process different payment types differently.
5. A simple example is a parent class Animal with method sound(). Child classes Dog and Cat can override sound() differently.
6. When dog.sound() prints “Bark” and cat.sound() prints “Meow”, the same method behaves differently. This demonstrates polymorphism in Python.

10. What is data hiding? How is it achieved in Python?

1. Data hiding is a concept of restricting direct access to important data in a class. It protects internal data from unauthorized modification.
2. In Python, data hiding is achieved using private members inside a class. Private members are defined using double underscore prefixes.
3. Variables like __salary cannot be accessed directly from outside the class. This improves security and maintains data integrity.
4. Python internally changes private variable names using name mangling. For example, __salary becomes _ClassName__salary internally.
5. Public methods are used to safely access or modify private data. Getter and setter methods are commonly used for this purpose.
6. For example, an Employee class may hide __salary and provide get_salary() method to access it safely. This prevents direct manipulation of sensitive data.

(5 – marks)

1. Explain Python scope resolution (LEGB rule) with example.

1. Python uses the LEGB rule to search variables in a program. LEGB stands for Local, Enclosing, Global, and Built-in scope.
2. Local scope contains variables declared inside a function. These variables can only be accessed within that function.
3. Enclosing scope exists in nested functions where inner functions access outer function variables. It is useful in nested programming.
4. Global scope contains variables declared outside all functions. These variables are accessible throughout the program.
5. Built-in scope contains predefined Python functions like print() and len(). Python checks this scope at the end.
6. Python searches variables in the order Local, Enclosing, Global, and Built-in. This order avoids confusion between variables with same names.
7. Example: if x=10 globally and x=20 locally, Python prints 20 inside the function and 10 outside. This proves local scope gets higher priority.

2. Write a Python program to demonstrate use of a constructor.

```
class Student:
  def __init__(self, name, marks):
    self.name = name
    self.marks = marks

  def display(self):
    print("Student Name:", self.name)
    print("Student Marks:", self.marks)

s1 = Student("Rahul", 90)
s1.display()
```

3. Explain single inheritance with a Python example.

1. Single inheritance is a type of inheritance where one child class inherits from one parent class. It is the simplest form of inheritance in Python.
2. The child class can use methods and properties of the parent class directly. This promotes code reusability and reduces duplication.
3. Inheritance creates a relationship between parent and child classes. The child class can also add its own methods.
4. Single inheritance improves maintainability because common code is written once. Any changes in the parent class automatically affect the child class.
5. Example: a Parent class may contain parentMethod() while Child class contains childMethod(). The Child object can access both methods.
6. Python uses class Child(Parent) syntax to inherit properties. The child class automatically receives accessible parent features.
7. Single inheritance is commonly used to build hierarchical and reusable programs. It makes object-oriented programming more organized and efficient.

4. Describe encapsulation and its importance in OOP.

1. Encapsulation is an OOP concept that combines data and methods into one unit called a class. It protects data from direct external access.
2. In encapsulation, private variables are hidden inside the class. Public methods are used to access or modify them safely.
3. Python achieves encapsulation using private members with double underscores. These members cannot be directly accessed outside the class.
4. Encapsulation improves data security and prevents unauthorized changes. It helps maintain data integrity in programs.
5. It also improves maintainability because internal implementation can change without affecting other code. This makes programs flexible and organized.
6. Getter and setter methods are commonly used with encapsulation. They provide controlled access to private data.
7. Example: an Employee class may hide __salary and allow access through methods only. This protects important information from misuse.

5. Write a short note on function overloading in Python.

1. Function overloading means using the same function name with different parameters. Different languages support this feature in different ways.
2. Python does not support true function overloading directly. It is dynamically typed and resolves methods during runtime.
3. If multiple functions with the same name are written, the latest definition replaces previous ones. Therefore, traditional overloading is not possible.
4. Python achieves similar behavior using default arguments and variable-length arguments. One function can handle different inputs in this way.
5. Function overloading improves flexibility and reduces the need for many function names. It also makes code shorter and easier to manage.
6. Example: add(a,b=0) can work with one or two arguments. This creates behavior similar to function overloading.
7. Overloading-like behavior helps write reusable and flexible programs in Python. It supports cleaner and more understandable code structure.

6. Explain method overriding with suitable example.

1. Method overriding is a feature where a child class provides its own version of a parent class method. Both methods have the same name and parameters.
2. It is an example of runtime polymorphism in Python. The method execution depends on the object calling it.
3. Overriding allows child classes to change parent class behavior according to requirements. This improves flexibility and code reusability.
4. When the child class defines the same method, the parent method gets overridden. The child version is executed instead of the parent version.
5. Example: a parent class Animal may have sound() method. A child class Dog can override it to print “Bark”.
6. If a Dog object calls sound(), the child class method runs automatically. This shows how overriding changes method behavior.
7. Method overriding is useful in applications where similar actions need different implementations. It makes object-oriented programs more dynamic and efficient.

7. Differentiate between class variables and instance variables.

1. Class variables are shared by all objects of a class. Instance variables are unique for each object.
2. Class variables are declared inside the class but outside methods. Instance variables are usually declared inside constructors using self.
3. Changing a class variable affects all objects of the class. Changing an instance variable affects only that specific object.
4. Class variables are used for common properties shared by all objects. Instance variables store data specific to each object.
5. Class variables can be accessed using class name or object reference. Instance variables are accessed mainly through object references.
6. Example: wheels=4 in a Car class is a class variable. brand="Toyota" for a car object is an instance variable.
7. Class variables save memory because they are shared among objects. Instance variables allow each object to maintain separate data values.

8. Explain multiple inheritance with syntax and example.

1. Multiple inheritance is a type of inheritance where one child class inherits from more than one parent class. The child receives features from all parent classes.
2. It allows combining functionalities of multiple classes into a single class. This improves code reuse and flexibility in programs.
3. Syntax of multiple inheritance is written as class Child(Parent1, Parent2). The child class can access methods from both parent classes.
4. Example: a class Mother may contain mother() method and class Father may contain father() method. A child class Son inherits from both classes.
5. The Son class can use methods and attributes of both Mother and Father classes. This demonstrates multiple inheritance in Python.
6. Python handles multiple inheritance using Method Resolution Order (MRO). MRO decides the order in which parent methods are searched.
7. Multiple inheritance is useful when a class requires features from different parent classes. It helps create more powerful and reusable applications.

9. Write a program to demonstrate polymorphism using functions.

```
class Dog:
  def sound(self):
    print("Dog barks")

class Cat:
  def sound(self):
    print("Cat meows")

def animalSound(animal):
  animal.sound()

d = Dog()
c = Cat()

animalSound(d)
animalSound(c)
```

10. Explain namespaces and variable lifetime in Python.

1. A namespace in Python is a container that stores names of variables, functions, and objects uniquely. It helps avoid conflicts between variable names.
2. Python mainly provides built-in, global, and local namespaces. Each namespace manages identifiers according to their scope.
3. The built-in namespace contains predefined functions like print() and len(). It exists as long as the Python interpreter runs.
4. The global namespace stores variables declared in the main program. It remains active until the program execution ends.
5. The local namespace is created when a function is called. It contains local variables and is destroyed when the function finishes execution.
6. Variable lifetime refers to the duration for which a variable exists in memory. Different scopes have different lifetimes in Python.
7. Built-in variables have the longest lifetime while local variables have the shortest lifetime. This helps Python manage memory efficiently.

(10 – marks)

1. Explain all Object-Oriented concepts (Encapsulation, Inheritance, Polymorphism, Abstraction) with examples.

1. Object-Oriented Programming uses objects and classes to organize programs. It improves modularity, maintainability, and code reuse.
2. Encapsulation means binding data and methods into one class. It protects data using private variables and controlled access methods.
3. Example of encapsulation is an Employee class hiding __salary variable. Getter and setter methods are used to access salary safely.
4. Inheritance allows one class to acquire properties of another class. It helps reduce code duplication and supports reusability.
5. Example of inheritance is a Child class inheriting methods from a Parent class. The child object can access parent methods directly.
6. Polymorphism means one method behaves differently for different objects. It provides flexibility and runtime method selection.
7. Example of polymorphism is Dog and Cat classes having different sound() methods. The same method name gives different outputs.
8. Abstraction hides internal implementation details from the user. Only important functionalities are exposed to users.
9. Example of abstraction is an ATM machine where users only see operations like withdraw or deposit. Internal banking processes remain hidden.
10. These four concepts form the foundation of OOP in Python. They help create secure, reusable, and scalable applications.

2. Write a Python program demonstrating inheritance types: a) Multilevel b) Hybrid.

1. Inheritance allows one class to use properties and methods of another class. It improves code reuse and program organization.
2. Multilevel inheritance happens when one derived class becomes the parent of another class. Features pass through multiple levels of inheritance.
3. Example of multilevel inheritance:

```
class A:
  def showA(self):
    print("Class A")

class B(A):
  def showB(self):
    print("Class B")

class C(B):
  def showC(self):
    print("Class C")

obj = C()
obj.showA()
obj.showB()
obj.showC()
```

4. In this example, class C inherits methods from both A and B. This demonstrates multilevel inheritance clearly.
5. Hybrid inheritance combines more than one inheritance type in a program. Python uses Method Resolution Order to manage such inheritance.
6. Example of hybrid inheritance:

```
class CEO:
  def ceo(self):
    print("CEO")

class Manager(CEO):
  def manager(self):
    print("Manager")

class Employee(Manager, CEO):
  def employee(self):
    print("Employee")
```

3. Explain types of inheritance (single, multiple, multilevel, hierarchical) with diagrams and examples.

1. Inheritance is an OOP feature where one class acquires properties and methods of another class. It promotes code reuse and hierarchical relationships.
2. Single inheritance involves one child class inheriting from one parent class. Example diagram: Parent → Child.
3. Example of single inheritance is a Child class inheriting methods from Parent class. The child object can access parent features directly.
4. Multiple inheritance occurs when one child class inherits from multiple parent classes. Diagram example: Parent1 + Parent2 → Child.
5. Example of multiple inheritance is Son class inheriting from Mother and Father classes. The child can use methods from both parents.
6. Multilevel inheritance involves inheritance through multiple levels of classes. Diagram example: Grandparent → Parent → Child.
7. Example of multilevel inheritance is Universe → Earth → India classes. The final child class accesses methods of all upper classes.
8. Hierarchical inheritance occurs when multiple child classes inherit from one parent class. Diagram example: Parent → Child1 and Child2.
9. Example of hierarchical inheritance is Manager class inherited by Employee1 and Employee2 classes. Both child classes share parent features.
10. These inheritance types improve flexibility, maintainability, and code organization in Python applications. They help build reusable object-oriented systems

4. Discuss encapsulation and data hiding in Python. Implement a class showing private variables and methods.

1. Encapsulation is an OOP concept that binds data and methods into one unit called a class. It improves security, maintainability, and data control.
2. Data hiding means restricting direct access to important variables and methods. Python achieves this using private members with double underscores.
3. Private variables cannot be accessed directly outside the class. Public methods are used to safely access or modify them.
4. Encapsulation protects data from accidental modification and improves program organization. It also helps achieve abstraction in applications.
5. Python internally changes private variable names using name mangling. This prevents direct external access to sensitive data.
6. Example program showing private variables and methods:

```
class Employee:
  def __init__(self, name, salary):
    self.name = name
    self.__salary = salary

  def __showSalary(self):
    print("Salary:", self.__salary)

  def display(self):
    self.__showSalary()

emp = Employee("Rahul", 50000)
print(emp.name)
emp.display()
```

7. In this example, __salary and __showSalary() are private members. They can only be accessed within the class methods.

5. Write a detailed note on polymorphism in Python, including operator overloading and method overriding.

1. Polymorphism means “many forms” where the same method or operator behaves differently in different situations. It increases flexibility and reusability in programs.
2. Python supports polymorphism mainly through method overriding and operator overloading. Different objects respond differently to the same operation.
3. Method overriding occurs when a child class provides its own version of a parent class method. This is an example of runtime polymorphism.
4. Example of overriding is Animal class having sound() method and Dog class overriding it with different behavior. The child method runs during execution.
5. Operator overloading means operators work differently for different data types. For example, + adds numbers but joins strings.
6. Python internally uses special methods like **add**() for operator overloading. This allows operators to behave according to object type.
7. Polymorphism improves flexibility because one interface supports multiple behaviors. It also reduces complexity in large applications.
8. Runtime polymorphism is decided during program execution based on object type. This makes Python programs dynamic and adaptable.
9. Built-in functions like len() also demonstrate polymorphism by working with strings, lists, and tuples differently. The same function gives different outputs.
10. Polymorphism is an important OOP feature for creating reusable and maintainable applications. It supports efficient and scalable software development.

6. Explain Python namespaces and scopes in detail with examples of LEGB rule.

1. A namespace in Python is a container that stores variable and function names uniquely. It prevents conflicts between identifiers in a program.
2. Python mainly provides built-in, global, and local namespaces. Each namespace manages variables according to their scope.
3. Built-in namespace contains predefined functions like print(), range(), and len(). It exists as long as the interpreter runs.
4. Global namespace stores variables declared outside all functions in the program. These variables remain until program execution ends.
5. Local namespace is created whenever a function is called. It stores local variables and disappears after function execution.
6. Scope refers to the region where a variable can be accessed in a program. Python follows the LEGB rule for scope resolution.
7. LEGB stands for Local, Enclosing, Global, and Built-in scope. Python searches variables in this exact order.
8. Local scope contains variables inside functions while enclosing scope exists in nested functions. Global scope contains module-level variables.
9. Example: if x=10 globally and x=20 locally, Python prints 20 inside the function because local scope has higher priority. Outside the function, it prints 10.
10. Namespaces and scopes improve memory management and avoid variable conflicts in Python applications. They make programs organized and easier to debug.

7. Design a class Student with attributes and methods. Demonstrate object creation and constructor usage.

1. A class is a blueprint used to create objects in Python. Objects store data and perform actions using methods.
2. The Student class can contain attributes like name, roll number, and marks. Methods are used to display student information.
3. Constructors initialize object data automatically during object creation. In Python, constructors are written using **init**() method.
4. The self keyword refers to the current object of the class. It helps each object maintain its own data separately.
5. Example program demonstrating Student class and constructor:

```
class Student:
  def __init__(self, name, rollno, marks):
    self.name = name
    self.rollno = rollno
    self.marks = marks

  def display(self):
    print(self.name, self.rollno, self.marks)

s1 = Student("Rahul", 101, 90)
s1.display()
```

6. In this example, s1 is an object created from Student class. The constructor initializes all attribute values automatically.
7. The display() method prints student details stored in the object. This demonstrates object creation and constructor usage in Python.

8. Write a Python program to implement multilevel inheritance and explain the output.

1. Multilevel inheritance occurs when one derived class becomes the parent of another class. Features are inherited through multiple class levels.
2. It helps reuse code across many related classes. Python supports multilevel inheritance using normal inheritance syntax.
3. Example program for multilevel inheritance:

```
class A:
  def showA(self):
    print("Class A")

class B(A):
  def showB(self):
    print("Class B")

class C(B):
  def showC(self):
    print("Class C")

obj = C()
obj.showA()
obj.showB()
obj.showC()
```

4. In this example, class B inherits from A and class C inherits from B. Therefore, class C gets methods from both parent classes.
5. The object obj of class C can access showA(), showB(), and showC() methods. This demonstrates inheritance across multiple levels.
6. The output prints “Class A”, “Class B”, and “Class C” in sequence. This proves inherited methods are available in the final child class.
7. Multilevel inheritance improves reusability and reduces code duplication in applications. It also creates hierarchical relationships between classes.

9. Compare procedural programming vs object-oriented programming with suitable examples.

1. Procedural programming focuses on functions and step-by-step procedures. Object-oriented programming focuses on classes and objects.
2. In procedural programming, data and functions are separate from each other. In OOP, data and methods are combined inside classes.
3. Procedural programming follows a top-down approach for solving problems. OOP mainly follows a bottom-up approach using reusable objects.
4. OOP provides features like encapsulation, inheritance, polymorphism, and abstraction. Procedural programming does not directly support these advanced concepts.
5. Procedural programs become difficult to manage in large applications. OOP improves maintainability, scalability, and flexibility for complex systems.
6. In procedural programming, functions are reused separately in programs. In OOP, classes and objects are reused through inheritance and polymorphism.
7. Security is lower in procedural programming because data is openly accessible. OOP provides better security using encapsulation and data hiding.
8. Procedural programming is suitable for small and simple applications. OOP is more suitable for large and real-world applications.
9. Example of procedural programming is writing separate functions for student records. Example of OOP is creating a Student class with attributes and methods.
10. OOP improves code reusability and organization while procedural programming is easier for beginners. Both approaches are useful depending on application requirements.

10. Develop a Python program demonstrating polymorphism using classes and functions. Explain its working.

1. Polymorphism means the same method or function behaves differently for different objects. It improves flexibility and code reuse in Python.
2. Python supports polymorphism using classes, functions, and method overriding. Different objects respond differently to the same function call.
3. Example program demonstrating polymorphism:

```
class Dog:
  def sound(self):
    print("Dog barks")

class Cat:
  def sound(self):
    print("Cat meows")

def animalSound(animal):
  animal.sound()

d = Dog()
c = Cat()

animalSound(d)
animalSound(c)
```

4. In this program, both Dog and Cat classes contain a method named sound(). The same method behaves differently for each object.
5. The function animalSound() accepts any object and calls its sound() method. The output depends on the object passed to the function.
6. When d is passed, the program prints “Dog barks”. When c is passed, the program prints “Cat meows”.
7. This demonstrates runtime polymorphism because behavior is decided during execution. Polymorphism makes programs flexible, reusable, and easier to maintain.